

Autism Eye Tracking System

Project Meeting & Presentation Cheat Sheet

1. The "Elevator Pitch" (What is the project?)

"Our project is an Autism Screening tool that uses a standard webcam to analyze eye-tracking and gaze patterns. It presents the user with split-screen images (faces vs. objects) and uses computer vision and machine learning (a Random Forest classifier) to determine if their visual attention pattern aligns more with typical development or indicates a potential risk for Autism Spectrum Disorder (ASD)."

2. The Core Methodology (The Science)

Our methodology is based on the **Preferential Looking Paradigm**, a well-established method in ASD research (pioneered by institutions like the UCSD Autism Center). When presented with a split-screen showing social stimuli (faces) and non-social stimuli (objects side-by-side):

- **Typical pattern:** Naturally drawn to social information, looking at faces > 55% of the time.
- **Atypical (ASD risk) pattern:** Reduced social visual attention, looking at faces < 40-45% of the time.

We use a **10-image screening test** because it is long enough to gather statistically significant gaze data, but short enough to maintain the user's attention throughout the test.

3. The Technical Stack (How we built it)

- **Language:** Python
- **Computer Vision:** MediaPipe (Face Mesh for 468 precise 3D facial landmarks) and OpenCV (video capture).

- **Machine Learning:** Scikit-Learn (Random Forest Classifier).
- **User Interface (GUI):** CustomTkinter (for a modern, dark-mode medical application look).
- **Data Handling:** Pandas and NumPy (saving logs, extracting gaze features).

4. Key Features We Built

- **GUI Control Panel:** Fully functional application with live camera preview and device selection.
- **9-Point Calibration:** Maps eye position accurately to the screen coordinates.
- **Screening Test:** Displays stimuli fullscreen and tracks gaze automatically in the background.
- **Machine Learning Classifier:** Analyzes test data, returns a "Risk Score %", and categorizes the user.
- **Automated Reports:** Generates and saves printable text reports for the patient under their name.
- **Analytics:** Generates visual heatmaps and extracts gaze metrics (fixations, saccades, scanpath).

5. The ML Model & "Synthetic Data"

Because we lacked access to clinical patient logs, and existing public datasets (like Kaggle) use incompatible hardware (expensive infrared trackers) and different video tests, we generated **400 synthetic samples** to train the model.

What is "Mathematically Generated" Data?

Using Python's NumPy library, we wrote an algorithm to simulate test results based strictly on thresholds published in medical journals. For example, our algorithm constrained the 'Typical' group to randomly generate Face Attention Ratios between 55-85%, while constraining the 'Atypical' group to 15-45%. This allows us to train a robust Random Forest classifier without violating medical privacy or requiring clinical access.

Evaluation Results:

- **Accuracy:** 94.75% (using 10-fold cross-validation)
- **AUC-ROC:** 0.995
- **Top Feature:** Face Attention Ratio

6. Limitations & Future Work

(Be honest about limits—evaluators love this!)

- **Hardware Limit:** We use a standard webcam/phone. While excellent for left/right tracking (face vs. object), it is not as pixel-perfect as a \$2000 infrared eye tracker (e.g., Tobii).
- **Data Limit:** The ML model is a proof-of-concept trained on synthetic data based on research, not real hospital datasets.
- **Scope Limit:** It is a *screening tool*, not a diagnostic tool.



Q&A Prep: Potential Supervisor Questions

Q: "Why did you use MediaPipe instead of standard OpenCV Haar Cascades for eye tracking?"

A: "MediaPipe Face Mesh is much faster, lighter, and more accurate for real-time tracking. It gives us 468 precise 3D facial landmarks, which allows us to pinpoint the exact iris center and calculate the Eye Aspect Ratio (EAR) for blinking much more reliably than older methods."

Q: "How exactly do you calculate where the user is looking?"

A: "We extract the iris landmarks and calculate their center relative to the eye corners. During calibration, we map these extreme iris positions to the edges of the screen. Then, during the test, we normalize the iris position to determine if they are looking at the left half or the right half of the screen."